CS 4032 – Web Programming

National University
Of Computer and Emerging Sciences

Async/Await

Async + Await

- "Syntactic sugar" that wraps a function's return in a promise
- Allows code to "wait" for the thing to return.
- `async` does the same thing to functions that `then` does
  - It wraps the return value in a `Promise` whose resolved value is the return value
- `await` halts execution of the code until the `Promise` is resolved and then returns the resolved value of the promise

.then/.catch vs async/await

Using the .then/.catch chaining

```
let newPromise = someFunction()

newPromise
    .then(func1)
    .then(func2)
    .then(func3)
    .catch(errHandler);
```

Using async/await

```
try {
        let a = await
someFunction();
        let b = await func1(a);
        let c = await func2(b);
        let d = await func3(c);
} catch(err) {
        errHandler(err);
}
```

Note: don't overuse `await`. Only include it before the function call when there is code to be `awaited`

What if?

```
let myBtn = qs('button:nth-child(1)');
while (!myBtn.clicked) {
  // twiddle our thumbs
}
console.log('"Finally Been Clicked", starring Drew Barrymore');

let myBtn2 = qs('button:nth-child(2)');
while (!myBtn2.clicked) {
  // twiddle our thumbs
}
console.log('"Click 2", never coming soon to a theater near you');
```

What if? (with Promises)

```
function firstBtnClick() {
    return new Promise(function (resolve) {
        let myBtn = qs('button:nth-child(1)');
        myBtn.addEventListener('click', resolve);
    });
}

function nextBtnClick() {
    return new Promise(function (resolve) {
        let myBtn = qs('button:nth-child(2)');
        myBtn.addEventListener('click', resolve);
    });
}

firstBtnClick()
    .then(() => { console.log('"Finally Been Clicked", starring Drew Barrymore'); })
    .then(nextBtnClick)
    .then(() => { console.log('"Click 2", never coming soon to a theater near you'); });
```

What if? (with Promises + A Little Syntactic Sugar)

```
function firstBtnClick() {
    return new Promise(function (resolve) {
        let myBtn = qs('button:nth-child(1)');
        myBtn.addEventListener('click', resolve);
    });
}

function nextBtnClick() {
    return new Promise(function (resolve) {
        let myBtn = qs('button:nth-child(2)');
        myBtn.addEventListener('click', resolve);
    });
}

await firstBtnClick();
console.log('"Finally Been Clicked", starring Drew Barrymore');
await nextBtnClick();
console.log('"Click 2", never coming soon to a theater near you');
```

Mind. Blown.

```
await firstBtnClick();
console.log('"Finally Been Clicked", starring Drew Barrymore');
await nextBtnClick();
console.log('"Click 2", never coming soon to a theater near you');
```

Async/Await

"Syntactic sugar" that wraps a function's return in a promise
Allows code to "wait" for the thing to return.

```
async function sayHelloAsync(name) {
  return "Hello " + name;
}

console.log(sayHelloAsync("dubs")); // Promise <pending>
let message = await sayHelloAsync("dubs");
console.log(message); // "Hello dubs"
```

- `async` does the same thing to functions that `then` does
  - It wraps the return value in a Promise whose resolved value is the return value
- `await` halts execution of the code until the Promise is resolved and then returns the resolved value of the promise

Back to that Pizza

```
function orderExecutor(resolve, reject) { // reject not required here
    console.log('making our pizza...');
    setTimeout(resolve, 5000);
}

await new Promise(orderExecutor);
console.log('eating pizza!');
```

Back to that Pizza

We can pass a value to resolve...

```
function orderExecutor(resolve, reject) {
    console.log('making our pizza...');
    setTimeout(function() {
        resolve("Here's your pizza!");
    }, 5000);
}

let pizza = await new Promise(orderExecutor);
console.log(pizza);
```

That value gets passed to the function passed into await

Back to that Pizza

```
async function eat(value) {
    return value + ", and now it's gone";
}

let pizza = await new Promise(orderExecutor);
let eatingResult = await eat(pizza);
console.log(eatingResult);
```

Back to that Pizza

You can also return other promises, which halt the execution of the next then callback until it's resolved

```
function eatExecutor(resolve, reject) {
    console.log('eating our pizza...');
    setTimeout(resolve, 3000);
}

async function eat() { // don't need async here...why not?
    return new Promise(eatExecutor);
}

let pizza = await new Promise(orderExecutor);
let eatingResult = await eat(pizza);
console.log('Paying the bill!');
```

Still Asynchronous… or is it?

```
function orderExecutor(resolve, reject) {
    console.log('Pizza ordered...');
    setTimeout(function() {
        resolve("Here's your pizza!");
    }, 3000);
}

let pizza = await new Promise(orderExecutor);
console.log(pizza);
console.log('Waiting around');
```

In what order do these log statements appear in the console?

Rejecting a Pizza

```
function orderExecutor(resolve, reject) { // MUST have both parameters passed in
    console.log('Pizza ordered...');
    setTimeout(function() {
        reject("Ran outta cheeese. Can you believe it?");
    }, 2000);
}

try {
    let pizza = await new Promise(orderExecutor);
    console.log("Woohoo, let's eat!");
} catch (error) {
    console.log(error);
}
```

Error handling with `async/await`

For error-handling with async/await, you must use try/catch instead of .then/.catch

The catch statement will catch any errors that occur in the then block (whether it's in a Promise or a syntax error in the function), similar to the .catch in a fetch promise chain

When Do I Need the Keyword `async`?

For any function that is `await`'d, but that doesn't return a promise (although it'll still work to add `async` even if it does)

```
async function eat(value) {
    return value + ", and now it's gone";
}
let pizza = await new Promise(orderExecutor);
let eatingResult = await eat(pizza); // don't need to do this. Why not?
console.log(eatingResult);
```

For any function that that uses `await` in its implementation

```
async function orderPizza() {
    let pizza = await new Promise(orderExecutor);
    return 'Done!';
}
console.log(await orderPizza());
console.log('What now?');
```

Exercise 1: Practice with Async/Await

- Given the provided JavaScript here (js), replace all usage of `.then/.catch` in the code so that the output is the same. You will need to define at least one async function.
- Use the console to check your work

(solution - don't peek yet ▲)

JavaScript Objects

- Data format that represents data as a set of JavaScript objects
- Can create a new object without creating a `class`
- Made up of field name/value pairs (basic structure shown below)

```
let myobj = {
  fieldName1: value1,
  ...
  fieldNameN: valueN
};
```

Example of Accessing Values

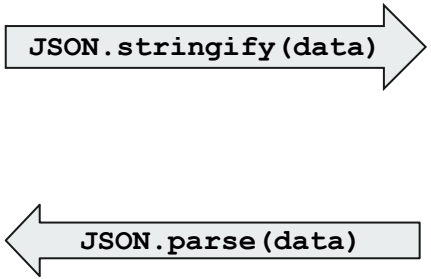
```
let data = {

        'course': 'cse154',

        'quarter': 'wi',

        'year': 2022,

        'university': 'uw',

        'grade-op': [4.0, 3.7, 2]

}
```

```
// access 'cse154' in 2 ways
data.course;
data['course'];

// access the grade 3.7
data['grade-op'][1];

// what about this?
data.grade-op[1];
```

Browser JSON methods

Method	Description
`JSON.parse(arg)`	converts JSON string into Javascript object
`JSON.stringify(arg)`	converts a Javascript object into JSON string

```
let data = {
        'course': 'cse154',
        'quarter': 'wi',
        'year': 2022,
        'university': 'uw',
        'grade-op':     [4.0,
3.7, 2]
}
```




```
"{"course":"cse154","qu
arter":"wi","year":2022
,"university":"uw","gra
de-op":[4,3.7,2]}"
```

Exercise 2: Practice with JS

Given the JavaScript Object returned from the CSE 154 Zoo API (found here), complete the table below mapping JavaScript code to its value/the value to it's JavaScript code.

JavaScript	Value
`data.company`	
`data['company']`	
	`'jjorg1@i2i.jp'`
`data['ticket-prices'].adult`	
	`5.95`
`data.ticket-prices`	
`data.age-median`	
	`20`